



Figure 1: The Jackal robot (clearpathrobotics.com).

Assignment 4

Simon Leonard

Due: April 14th 2026

1 Introduction

In this assignment, you will implement an extended Kalman (EKF) filter to estimate the location of a mobile robot (Clearpath Robotics Jackal) on a simulated rugged terrain. The simulation uses the Gazebo environment.

In the simulation, the robot is equipped with a GPS sensor to determine the robot coordinates and an inertial measurement unit (IMU) to measure its orientation.

The simulation requires a fair amount of computational resources and using a virtual machine is not recommended with a single CPU.

The EKF engine is based on the Orocos Bayesian Filtering Library (BFL) and your task will to:

1. Determine and implement the 3D process model of the robot (2.5pt).
2. Determine and implement the observation model for both sensors (GPS and IMU) (2.5pt).
3. Determine and implement all the Jacobians (system and sensors) (2.0pt).
4. Determine process and sensor covariance matrices (filter tuning) (2.0pt).
5. Expert Challenge (1pt).

The EKF node publishes the posterior state estimate robot's position on the topic `/posterior`. You can compare the posterior estimate to the true position by obtaining it from the simulator (with a real robot you cannot do this) with the command `ign topic -e -t /model/robot/pose`. In this assignment, the IMU measurements are available on the topic `/sensors/imu_0/data` and the GPS on the topic `/sensors/gps_0/fix`.

2 Installation

Before installing the package, you will need to install several ROS packages by using the following commands. You do not need to install these on Wyman workstations.

```
[sleonard@percival ~]$ sudo apt-get update
[sleonard@percival ~]$ sudo apt-get upgrade
[sleonard@percival ~]$ sudo apt-get install ros-humble-clearpath*
```

Install the package and dependencies with the following commands

```
[sleonard@merliver ~]$ wget www.cs.jhu.edu/~sleonard/sbr-a4.sh
[sleonard@merliver ~]$ chmod 755 sbr-a4.sh
[sleonard@merliver ~]$ ./sbr-a4.sh
```

and to run the assignment use the following commands

```
[sleonard@merliver ~]$ ros2 launch assignment4 assignment4.launch.py
```

The package contains several directories and files, but all your implementation is limited to the files `ekf_models.hpp` and `ekf_models.cpp`.

2.1 Deliverables

1. Your entire modified `assignment4.tgz` package that includes the code of your models.

To prepare your submission, use the command provided at the end of the document.

3 Teleoperation

To teleoperate the robot by using your keyboard or the buttons in the window illustrated in Fig. 2. But first, you need to start the simulation by pressing the “Play” button in the bottom left corner. You can also pause the simulation by pressing it again. After the starting simulation, the robot should (hopefully) fall gracefully to the ground.

To move the robot, you need to use the buttons to move the robot or use the keyboard by selecting the keyboard tab and using the keys.

Important Note

You must send one teleoperation command for the EKF to initialize. The EKF will start its calculations after that.

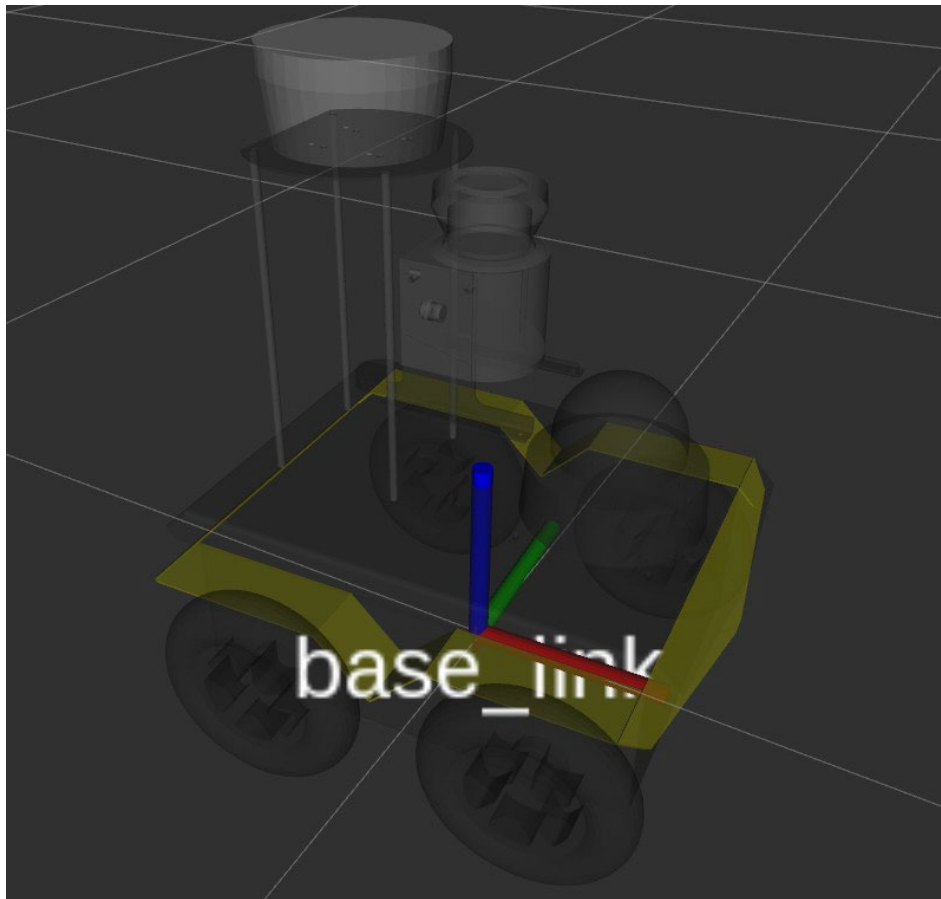


Figure 2: The coordinate system of the Jackal.

4 Extended Kalman Filter Localization

An EKF is composed of system prediction and correction and you must implement all the functions associated with them. The 6D state of the robot is defined by

$$\mathbf{s}_t = [x \ y \ z \ r \ p \ y]^T$$

where (x,y,z) are the 3D coordinates of the robot and (r,p,y) are the robot's *roll*, *pitch* and *yaw* (RPY). Roll refers to the 3D rotation about the X (red) axis, pitch refers to the 3D rotation about the Y (green) axis and yaw refers to the 3D rotation about the Z (blue) axis. Note that almost all sensors in ROS use quaternions to measure orientations whereas the EKF will use RPY. In this case, RPY and quaternions represent the same thing: the orientation of the Jackal with respect to a world frame, but they do it with a different convention. In this assignment, the orientation of the Jackal is given by $R = R_x(r)R_y(p)R_z(y)$, where R_x , R_y and R_z are rotations about the X, Y and Z axes. This is very important because it simplifies your calculation *a lot*. The coordinate frame used for the localization is named base link and is illustrated in Fig. 2.



Figure 3: The Jackal in the Gazebo environment.

4.1 System Model

In this course, we have mainly presented localization methods for planar robots (a mobile robot that moves on a 2D plane). In this assignment, however, the robot can move up and downhill and therefore all six degrees of freedom must be accounted for.

Your first task is to implement the function $\mathbf{f}$ that models the system's equation

$$\mathbf{s}_{t+1} = \mathbf{f}(\mathbf{s}_t, \mathbf{u}_t, \mathbf{w}_t)$$

where $\mathbf{u}_t = [v_t \ \omega_t]^T$ represents the external input of a linear v_t and angular ω_t velocities. The linear velocity v_t is along the X axis of the Jackal, whereas the angular velocity ω_t is about the Z axis of the Jackal. Once you have determined $\mathbf{f}$, implement it in the function `sys_evaluate_f`. You must also implement the 6x6 Jacobian in $A = \frac{\delta \mathbf{f}}{\delta \mathbf{s}}$ function `sys_evaluate_A`. You are free to use any software (Matlab, Mathematica,...) to derive your equations.

Finally, you must implement the system's covariance in the function `sys_evaluate_WMWt` based on the command sent to the robot. This step is known as "filter tuning" and you will need to experiment with different values and use your judgment.

4.2 Observation Models

The Jackal uses two sensors: A GPS to measure the longitude, latitude, and altitude and an IMU to measure the 3D orientation. Like the system model, you will have to experiment with different values for the covariance of the sensors in the function `meas_evaluate_R` and submit the covariance that work the best for you.

4.2.1 GPS

The GPS provides “fixes”. Each fix contains the robot’s latitude (λ), longitude (ϕ) and altitude (ψ). For our purpose, we want to convert these coordinates in a Cartesian frame (X-Y-Z). There are several transformations between geographic and Cartesian systems (i.e. World Geodetic System revision 84¹) but it is reasonable to assume that the earth is flat around the robot (linear model) and figure a way to map GPS data to the XYZ coordinate frame of the simulation. For this you might want to drive the robot around, collect data from the topics, and figure out the relationship between $\mathbf{z}_t$ and $\mathbf{s}_t$.

The geographic coordinates are related to the Cartesian coordinates by the functions

$$\begin{bmatrix} \lambda \\ \phi \\ \psi \end{bmatrix} = \mathbf{h}_{gps}(\mathbf{s})$$

Feel free to “cheat” by using the true state of the Jackal to determine $\mathbf{h}_{gps}$ by mapping the X-Y-Z coordinates of the Jackal to GPS measurements. The true position of the Jackal is available by the command `ign topic -e -t /model/robot/pose`

You must determine and implement $\mathbf{h}_{gps}$ in the function `meas_evaluate_gps` and implement the 3x3 Jacobian H_{gps} in the function `meas_evaluate_Hgps`.

4.2.2 IMU

The IMU sensor determines the orientation of the robot. Like the GPS, you must implement the function $\mathbf{h}_{imu}$ in `meas_evaluate_imu` and its 3x3 Jacobian H_{imu} in the function `meas_evaluate_Himu`. This part is relatively simple.

4.2.3 Observation Jacobian

The code in the assignment will assemble both Jacobians H_{GPS} and H_{IMU} into a single matrix

$$H = \begin{bmatrix} H_{GPS} & 0 \\ 0 & H_{IMU} \end{bmatrix}$$

Likewise, both measurements are assembled

$$\mathbf{h}(\mathbf{s}_t, 0) = \begin{bmatrix} \mathbf{h}_{GPS} \\ \mathbf{h}_{IMU} \end{bmatrix}$$

¹ https://en.wikipedia.org/wiki/World_Geodetic_System

5 Expert Challenge

Your EKF will be tested by driving the Jackal around the canyon and compare the posterior estimate to the true position of the Jackal provided by Gazebo. To evaluate the accuracy, you can access the position/orientation of the frame base link from the command

```
ign topic -e -t /model/robot/pose
```

We will drive the Jackal around the landscape, and you will succeed in the challenge if your average XYZ position error $\|ee\|_2$ is less than 0.5 meter. For this you will need to drive the robot around and collect the data from the simulation and your EKF to determine errors at various points.

6 Submission

You need to create and submit your assignment4 package by using the following command.

```
sleonard@swami:~$ tar cvfz ~/assignment4.tgz -C ~/sbr_workspace/src/sbr assignment4
```

This will create the file assignment4.tgz. You must submit this file on Canvas. A penalty of 1 mark will be deducted for submissions that are not packaged accordingly or do not build cleanly.

Reminder: By submitting your assignment, you acknowledge that you have read the JHU Academic Misconduct Policy and that you are the author of 100% of the code submitted. Furthermore, distributing or sharing your assignments (whereas online, with classmates or students that will take this course in future semesters) is not authorized and will be severely sanctioned.